

Задача 1. Архитектура ядра «Умного дома»

Контекст:

Вы разрабатываете управляющий модуль для контроллера умного дома. Модуль должен работать непрерывно и выполнять три разнородных процесса.

Техническое задание (ТЗ):

Программа должна одновременно запускать и координировать 3 функции:

1. **sensor_polling()** (Опрос датчиков): Каждые 0.5 секунд опрашивает 3 удаленных датчика (температура, влажность, CO2). Имитируйте опрос как сетевую паузу продолжительностью от 100 до 200 мс (случайно для каждого датчика). Выводите показания в консоль.
2. **user_interface()** (Интерфейс управления): Программа должна непрерывно ожидать ввод команд от пользователя в консоли (например, команды `status`, `reboot`, `exit`). Ввод команд **не должен** прерывать или задерживать вывод данных с датчиков. При вводе команды сервер должен мгновенно выводить ответ.
3. **backup_logs()** (Резервное копирование): Раз в 5 секунд система должна сохранять текущую историю логов в текстовый файл `backup.txt`.

Особое условие (Ограничение):

- Для имитации долгой записи на медленный диск в функции `backup_logs()` **запрещено** использовать асинхронные библиотеки работы с файлами. Разрешено использовать только классический синхронный контекстный менеджер `with open()` и блокирующую задержку `time.sleep(1.5)`.

Критерий успешности:

Во время выполнения записи логов (когда отрабатывает задержка в 1.5 секунды), опрос датчиков и реакция на ввод пользователя **не должны замирать**. Программа должна оставаться отзывчивой.

Задача 2. Распределенный Банкомат и Асинхронный Аудит

Контекст:

Разрабатывается прототип транзакционного ядра для обслуживания единого банковского счета из разных точек сети.

Техническое задание (ТЗ):

1. **Общий ресурс:** Создайте переменную `bank_balance` с начальным значением `1000` рублей.
2. **Многопоточное списание:** Создайте функцию `withdraw_cash(amount, atm_name)`, которая проверяет, достаточно ли средств, делает паузу через `time.sleep(0.1)` (имитация проверки PIN-кода банкоматом) и уменьшает баланс на указанную сумму.
3. **Симуляция нагрузки:** Одновременно запустите 5 независимых потоков, имитирующих разные банкоматы. Каждый банкомат пытается списать со счета случайную сумму от 100 до 300 рублей.
4. **Асинхронный логгер транзакций:** Каждое действие (успешное списание или отказ из-за нехватки средств) должно отправлять отчетную строку в единую асинхронную очередь. Отдельная фоновая асинхронная задача должна вычитывать данные из этой очереди и «отправлять по сети на центральный сервер» (выводить на экран с асинхронной задержкой `await asyncio.sleep(0.2)`).

Критерий успешности:

- Баланс счета ни при каких условиях **не должен уйти в минус**, несмотря на то, что проверка баланса и списание разделены паузой.
- Студент должен продемонстрировать понимание концепции *Race Condition* (*Состояния гонки*) и решить её встроенными инструментами синхронизации, а также корректно связать синхронные потоки банкоматов с асинхронной очередью логгера.

Задача 3. Асинхронный Парсер с тяжелой CPU-аналитикой

Контекст:

Вам необходимо разработать утилиту, которая скачивает текстовые веб-страницы и проводит глубокий математический/криптографический анализ их содержимого.

Техническое задание (ТЗ):

1. **Сетевой этап (I/O-bound):** Напишите функцию, которая имитирует скачивание 10 страниц из интернета. Скачивание каждой страницы занимает ровно 1 секунду. Задача должна выполняться конкурентно (все страницы должны скачаться суммарно примерно за 1 секунду).
2. **Вычислительный этап (CPU-bound):** Как только страница «скачана», над её содержимым должен запуститься тяжелый вычислительный алгоритм `analyze_data()`. Имитируйте его «тяжесть» с помощью математического

цикла (например, подсчет суммы квадратов чисел от 1 до 20 000 000).
Процессор должен быть загружен на 100%.

Критерий успешности:

- Если запустить вычисления прямо внутри асинхронного Event Loop, он заблокируется, и скачивание остальных страниц остановится.
- Если запустить вычисления в обычных потоках (**threading**), они упрутся в **GIL (Global Interpreter Lock)** и будут выполняться строго по очереди на одном ядре процессора, не давая выигрыша в скорости.
- **Цель:** Найти архитектурное решение, при котором сетевые задачи не блокируются вычислениями, а сами вычисления распределяются по нескольким ядрам процессора. Измерьте общее время работы программы — оно должно быть оптимизировано.